Augmenting LLM Code Translation with Compiler Analysis for C to Triton Kernel Generation

Xiao Qin
University of Leeds
Leeds, United Kingdom
ppnh4382@leeds.ac.uk

Chunwei Xia
University of Leeds
Leeds, United Kingdom

Zheng Wang
University of Leeds
Leeds, United Kingdom

ABSTRACT

Emerging programming models like Triton enable developers to better exploit modern accelerators, but translating legacy code to Triton remains challenging. While Large Language Models (LLMs) show promise for code translation, they often generate incorrect or suboptimal implementations due to a lack of precise parallelization reasoning. We present TritonPilot, a compiler-assisted LLM framework for translating C loop nests to Triton kernels. TritonPilot extracts parallelization properties using compiler-based analysis and encodes them as structured analysis facts in the LLM prompt, enabling correct and performance-aware parallel code synthesis. A profiling feedback loop further improves performance by feeding GPU hardware utilization metrics back to the LLM for iterative optimization. Across PolyBench/C and TSVC kernels, TritonPilot achieves near-perfect correctness (100% and 99%), improving over LLM-only baselines. On TSVC at realistic data sizes, the generated Triton kernels achieve 1.02× median performance vs. OpenMP GPU offloading compiled by NVIDIA's production compiler on the same GPU. Profiling feedback improves 15 of 30 PolyBench kernels, with gains up to 47× on individual kernels. We further demonstrate generalization on Rodinia and ECP proxy applications. These results show that combining compiler analysis with hardware profiling feedback enables both correct and high-performance LLM-based parallel code generation.

CCS CONCEPTS

• **Software and its engineering** → **Compilers**; • **Computing methodologies** → *Parallel computing methodologies*.

KEYWORDS

LLM code generation, GPU kernels, Triton, polyhedral analysis

ACM Reference Format:
Xiao Qin, Chunwei Xia, and Zheng Wang. 2026. Augmenting LLM Code Translation with Compiler Analysis for C to Triton Kernel Generation. In *Proceedings of Workshop on AI for High Performance Compiler Construction (AI4HPCC '26).* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

AI4HPCC '26, July 6–9, 2026, Belfast, Northern Ireland, UK
2026. ACM ISBN 978-x-xxxx-xxxx-x/YY/MM. . . $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Accelerators such as GPUs are now essential for high-performance scientific computing, yet much existing software is still written in legacy C code that is not designed for parallel execution on modern accelerators. Translating these legacy programs to accelerator-friendly implementations is challenging, as it requires identifying parallelism, restructuring computation, and managing synchronization, tasks that require expertise in programming models and hardware.

Emerging programming models such as Triton [1] simplify GPU programming by providing higher-level abstractions for writing efficient kernels. However, mapping legacy C code to Triton still requires careful reasoning about loop-level parallelism, data dependencies, and execution order. Developers must determine which loop dimensions can be safely parallelized, how memory accesses should be organized, and when synchronization is required - decisions that are subtle and error-prone in practice.

Large Language Models (LLMs) have recently shown promise in code translation, including generating GPU kernels from high-level descriptions [2, 3]. However, LLMs lack precise reasoning about program semantics, especially data dependencies. As a result, they often produce incorrect or inefficient code, including unsafe parallelization that leads to data races, suboptimal kernel structures, and missing synchronization.

In this paper, we present TritonPilot, a system that augments LLM-based translation with compiler analysis. TritonPilot extracts parallelization properties from C loop nests using polyhedral analysis (PET) [4] and LLVM dependence analysis [5], and encodes them as structured guidance in the LLM prompt. This allows the model to generate correct and performance-aware Triton code without requiring it to perform dependence analysis itself.

We evaluate TritonPilot on a diverse set of benchmarks and application kernels on an NVIDIA RTX 3090 GPU. Across PolyBench/C and TSVC loop kernels, TritonPilot achieves near-perfect correctness (100% and 99%), significantly improving over LLM-only baselines. At realistic data sizes, the generated Triton kernels match the performance of OpenMP GPU offloading compiled by NVIDIA's `nvc` compiler (1.02× median on the same GPU). We further demonstrate generalization on kernels from Rodinia and ECP proxy applications. This paper makes the following contributions:

- A compiler-assisted LLM framework for translating legacy C loop nests to Triton, with a unified analysis module that encodes parallelization constraints as structured facts rather than pattern-specific templates.

- A profiling-guided optimization loop that feeds GPU hardware utilization metrics (from NVIDIA Nsight Compute) back to the LLM, enabling iterative performance improvement while preserving correctness.
- Empirical results showing that compiler analysis ensures correctness (100% on PolyBench/C) and that profiling feedback recovers the performance of hand-tuned prompt templates while maintaining generality.

2 OUR APPROACH

Figure 1 shows the TritonPilot workflow. Given a C source file, TritonPilot extracts loop nests, analyzes their parallelization properties, constructs a structured prompt, invokes an LLM to generate Triton code, and optionally refines performance using GPU profiling feedback.

2.1 Analysis and Prompt Construction

TritonPilot calls PET [4] once per kernel to extract a polyhedral representation (iteration domains, access relations, schedules). From this single invocation, it derives all parallelization constraints in a unified JSON schema: *(i)* for each loop dimension, whether it can be safely parallelized (by checking for cross-iteration dependences via ISL relation composition); *(ii)* WAR (write-after-read) dependences that require array cloning or sequential execution; *(iii)* reduction patterns (scalar accumulation + array read) mapped to Triton primitives; *(iv)* memory access patterns (stencil, triangular, cross-phase) that affect kernel structure. When polyhedral analysis fails (e.g., non-affine accesses), TritonPilot falls back to LLVM's `DependenceAnalysis`.

A single, pattern-agnostic function renders the analysis into structured text: it emits whatever constraints were found without pattern-specific templates. The prompt includes the C source, analysis facts (e.g., "`j` is safe; `t` has cross-phase deps"), and Triton coding rules. The LLM receives facts and decides the implementation strategy.

2.2 Generation and Iterative Refinement

TritonPilot invokes Claude Sonnet 4 [7] and validates the generated kernel against the C reference using identical inputs and FP32 tolerance checks. On failure, errors are classified (compilation, numerical, missing barriers, low parallelism) and a targeted retry prompt is issued, up to 10 attempts with a context reset at attempt 6.

2.3 Profiling-Guided Optimization

After correctness validation, TritonPilot optionally profiles the kernel with NVIDIA Nsight Compute (NCU), collecting SM throughput, memory throughput, occupancy, and cache hit rates. It classifies the bottleneck (*compute-bound*, *memory-bound*, or *latency-bound*) and feeds the metrics back to the LLM with bottleneck-specific optimization guidance. The optimized kernel is re-validated for correctness and kept only if faster. This loop runs up to 3 iterations, with NCU results cached when the implementation is unchanged.

Table 1: PolyBench/C results at 1× and 8× sizes. WA = with analysis, NA = no analysis, ST = single-threaded C, OMP = 80-thread OpenMP.

Metric	NA	WA	WA 8× (ST)
Pass rate	28/30 (93%)	30/30 (100%)	30/30
Median speedup	1.06×	2.07×	30.73×
Mean speedup	1.52×	2.58×	—
Kernels > 1×	14/28	20/28	21/25*

*5 sequential kernels timeout at 8×.

3 EVALUATION SETUP

Platforms. Experiments run on an NVIDIA RTX 3090 GPU (24 GB, 936 GB/s) with a dual-socket Xeon Gold 5218R CPU (80 cores). We use PyTorch 2.5.1, Triton 3.1.0, CUDA 12.4, and Claude Sonnet 4 [7].

Benchmarks. We evaluate on: (1) *PolyBench/C* [6]: 30 kernels at default sizes (N=60–120) and 8× scale; (2) *TSVC* [15]: 151 loop kernels at 32 MB and 3200 MB per array; (3) 8 application kernels from Rodinia [13] and ECP proxy apps [14].

Baselines. We compare against OMP GPU (OpenMP offloading via `nvc` on the same GPU), OMP CPU (80-thread OpenMP), and single-threaded C (`-O3`). For the ablation, we compare *with analysis* (WA) vs. *no analysis* (NA, LLM receives only C source).

4 EXPERIMENTAL RESULTS

4.1 PolyBench/C (30 Kernels)

Table 1 summarizes results. With analysis, all 30 kernels pass correctness; without it, two stencil kernels fail from parallelizing dimensions with carried dependences. At 1× sizes, the median speedup is 2.07× over single-threaded C; at 8× sizes, 30.73×. Top kernels: `heat_3d` (10.27×), `doitgen` (7.40×), `covariance` (5.40×), `deriche` (5.14×).

In the head-to-head comparison, 3 of 6 apparent NA wins are false: the unguided LLM parallelizes dimensions with carried dependences, producing code that runs faster but computes incorrect results (e.g., `lu`: output differences up to 10^{33}).

4.2 Profiling-Guided Optimization

Profiling feedback improved 20 of 28 passing kernels (Table 2 shows representative gains). The largest improvements occur on latency-bound kernels where the initial implementation has insufficient parallelism: the NCU metrics reveal low SM and memory utilization, and the LLM restructures the kernel to use more parallel resources. Correctness is preserved: each optimized kernel is re-validated, and attempts that break correctness are rejected.

4.3 TSVC (151 Kernels)

TritonPilot passes 150/151 TSVC kernels (99.3%), 120 on first attempt. At 32 MB per array, LLM-generated Triton

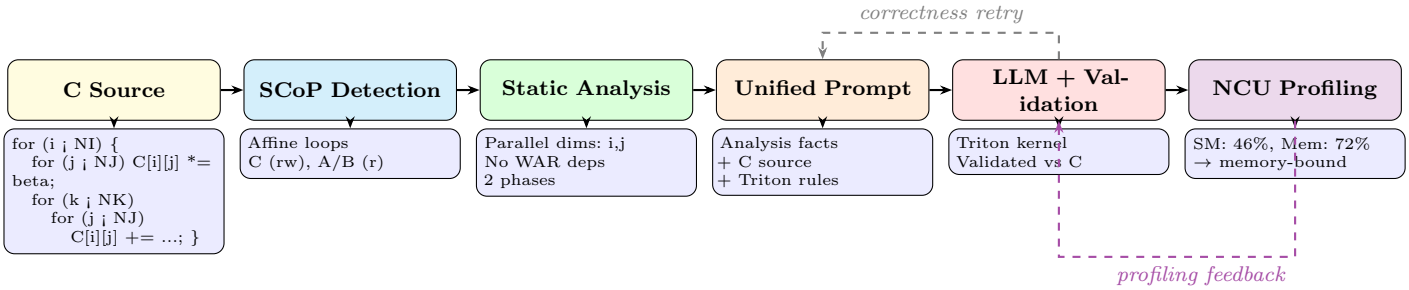

Figure 1: The TritonPilot workflow. Dashed arrows show iterative loops: the gray loop retries on correctness failures; the violet loop feeds NCU profiling metrics back to the LLM for performance optimization.

Table 2: Profiling feedback impact. "Before" = initial correct kernel; "After" = best after up to 3 NCU profiling iterations.

Kernel	Before	After	Bottleneck
covariance	0.15×	5.40×	latency
deriche	0.11×	5.14×	latency
3mm	0.12×	4.24×	latency
correlation	0.29×	3.38×	latency
adi	0.10×	2.07×	latency
gemver	0.16×	1.94×	latency
durbin	0.30×	1.47×	latency
fdtd_2d	0.85×	1.80×	latency
gemm	2.41×	3.10×	latency

Table 3: Application kernels: Triton vs. OMP GPU on the same GPU.

Source	Kernel	Tri/OMP GPU	WA/ST
Rodinia	SRAD	24.9×	24.9×
Rodinia	gaussian	249×	3.4×
miniMD	LJ force	6.8×	25.8×
miniFE	SpMV	3.1×	9.0×
Rodinia	lud	—[a]	13.2×

[a] `nvc` fails to compile this kernel.

achieves 67× over single-threaded C, 10.7× over 80-thread OpenMP CPU, and **1.02× vs. OpenMP GPU offloading** (`nvc -mp=gpu`) on the same GPU—matching production compiler quality.

4.4 Application Kernels

All 8 application kernels pass with analysis (Table 3 shows 5 representative results). On kernels with OMP GPU baselines, Triton is 3–249× faster than `nvc`'s offloading, due to Triton's block-level model avoiding per-timestep launch overhead.

5 RELATED WORK

LLM Code Generation for GPUs. Chen et al. [2] find frequent correctness issues in ChatGPT-generated CUDA kernels. KernelBench [3] reports 20–40% functional correctness for LLM-generated GPU kernels without guidance. Recent works address this via RL fine-tuning [16–18], search-based optimization [19–21], memory-augmented agents [22], and NPU-specific approaches [23, 24]. These focus on *how* to generate/optimize kernels; our work is orthogonal, addressing *what* parallelism is safe via compiler analysis, and could provide correctness constraints for these frameworks.

Polyhedral Compilation. PPCG [12] generates CUDA from affine loop nests but lacks Triton's block-level abstractions. TVM [9] and Halide [10] use auto-scheduling for tensor operations. TritonPilot uses polyhedral analysis to extract parallelism constraints, then delegates implementation to the LLM, handling non-affine constructs that polyhedral compilers cannot target.

6 CONCLUSION

We presented TritonPilot, which combines compiler analysis with LLM code generation for C-to-Triton translation. A single PET invocation derives all parallelization constraints, which are rendered as structured facts for the LLM. A profiling feedback loop using NCU metrics iteratively improves performance. TritonPilot achieves 100% correctness on PolyBench/C and 99% on TSVC, with generated Triton matching `nvc`'s OpenMP GPU offloading (1.02× on the same GPU). Profiling feedback improved 20 of 28 kernels, with gains up to 36× on individual kernels. These results suggest that compiler analysis and hardware profiling provide complementary signals that could serve as correctness constraints and performance rewards in RL-based or search-based kernel optimization frameworks.

REFERENCES

[1] P. Tillet, H. T. Kung, and D. Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proc. MAPL*, 2019.
[2] Y. Chen, S. Huang, D. Zheng, et al. ChatGPT for CUDA programming: Evaluating LLM-generated GPU code. *arXiv preprint arXiv:2404.12000*, 2024.
[3] A. Ouyang, S. Zheng, et al. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2404.12489*, 2024.
[4] S. Verdoolaege and T. Grosser. Polyhedral extraction tool. In *Proc. IMPACT*, 2012.
[5] C. Lattner and V. Adve. LLVM: A compilation framework for lifelong program analysis & transformation. In *Proc. CGO*, 2004.

[6] L.-N. Pouchet. Polybench/C: The polyhedral benchmark suite, version 4.2.1. https://web.cs.ucla.edu/~pouchet/software/polybench/, 2016.
[7] Anthropic. Claude Sonnet 4 model card. https://www.anthropic.com/claude, 2025.
[8] S. Verdoolaege. isl: An integer set library for the polyhedral model. In *Proc. ICMS*, 2010.
[9] T. Chen, T. Moreau, Z. Jiang, et al. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proc. OSDI*, 2018.
[10] J. Ragan-Kelley, C. Barnes, A. Adams, et al. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proc. PLDI*, 2013.
[11] U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan. A practical automatic polyhedral parallelizer and locality optimizer. In *Proc. PLDI*, 2008.
[12] S. Verdoolaege, J. Carlos Juega, A. Cohen, et al. Polyhedral parallel code generation for CUDA. *ACM TACO*, 9(4), 2013.
[13] S. Che, M. Boyer, J. Meng, et al. Rodinia: A benchmark suite for heterogeneous computing. In *Proc. IISWC*, 2009.
[14] M. A. Heroux, D. W. Doerfler, P. S. Crozier, et al. Improving performance via mini-applications. Sandia National Laboratories Technical Report SAND2009-5574, 2009.
[15] D. Callahan, J. Dongarra, and D. Levine. Vectorizing compilers: A test suite and results. In *Proc. Supercomputing*, 1988.
[16] W. Liu, J. Xu, Y. Li, et al. Dr. Kernel: Reinforcement learning done right for Triton kernel generations. *arXiv preprint arXiv:2602.05885*, 2026.
[17] A. Tehrani, Y. Emara, E. Wissam, et al. Fine-tuning GPT-5 for GPU kernel generation. *arXiv preprint arXiv:2602.11000*, 2026.
[18] H. Bai, L. Kong, X. Chen, et al. DICE: Diffusion large language models excel at generating CUDA kernels. *arXiv preprint arXiv:2602.11715*, 2026.
[19] T. Chen, Z. Ye, B. Xu, et al. AVO: Agentic variation operators for autonomous evolutionary search. *arXiv preprint arXiv:2603.24517*, 2026.
[20] S. Cao, Z. Mao, J. E. Gonzalez, and I. Stoica. K-Search: LLM kernel generation via co-evolving intrinsic world model. *arXiv preprint arXiv:2602.19128*, 2026.
[21] A. Bhattacharjee, H. Ping, S. V. Le, et al. OptiML: An end-to-end framework for program synthesis and CUDA kernel optimization. *arXiv preprint arXiv:2602.12305*, 2026.
[22] K. S. Dong, S. Modi, D. Nikiforov, et al. KernelBlaster: Continual cross-task CUDA optimization via memory-augmented in-context reinforcement learning. *arXiv preprint arXiv:2602.14293*, 2026.
[23] X. Cao, J. Zhai, P. Li, et al. AscendKernelGen: A systematic study of LLM-based kernel generation for neural processing units. *arXiv preprint arXiv:2601.07160*, 2026.
[24] Z. Wen, S. Shao, Z. Li, et al. AscendCraft: Automatic Ascend NPU kernel generation via DSL-guided transcompilation. *arXiv preprint arXiv:2601.22760*, 2026.